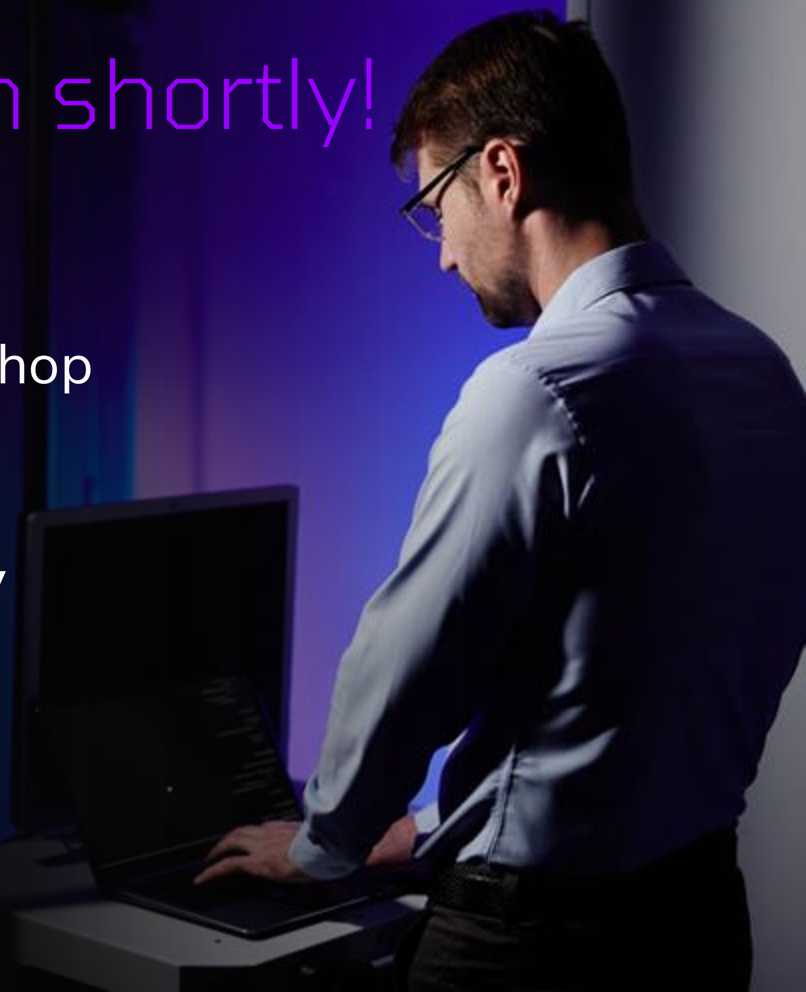
- The views and tools presented in this talk do not necessarily reflect the official position, practices, or endorsements of O.MG / Mischief Gadgets LLC.

# Hands on DuckyScript: An Introduction

# Welcome! We will begin shortly!



We are excited to have you at this workshop

Keep in mind this is meant for

**Officially Licensed Devices of DuckyScript Only**

(Hak5 Devices and O.MG Devices)

Please make sure your VM(s) are ready, and you have the training scripts!

If you don’t have them already please grab them from **https://da.gd/mischief**

# Facilitators

**wasabi**

Blue Teamer, Tinkerer, Security Researcher

**Toku**

Cloud Engineer of the Arcane

**01**

Red Teamer, Beer Enthusiast

# wasabi

## Tools and Infrastructure

Researcher and tinkerer

**Area of expertise**
IoT and Embedded, Cloud and Infrastructure

**Academic background**
Professor of Cyber Security

twitter.com/spiceywasabi
bsky.app/profile/spicywasabi.bsky.social

## Toku

**Workshop Presenter**

Cloud Engineer finding his way back to the physical world, dabbler in the arcane arts of Sisyphus

# 0lphorl³

## Tools and Testing

Red Teamer and Criminal Mastermind

**Area of expertise**

Offensive Security & Strategies, Logistics

**Academic background**

𝕏 @01p8or13

Over the next 4 hours we will teach you how to use O.MG devices to perform physical HID attacks against target systems

# Before we begin:

**Make sure you have the VM, Resources, and Drivers installed!**

**Optionally Connect to WiFi:**
SSID: HIDLAB
Pass: briskkey14
(Treat this network as hostile)

Files/Resources/ZIPs can be accessed at 192.168.1.10 or (files.lan)

# What are O.MG Devices
## (and what is Mischief Gadgets?)

> O.MG devices are covert USB peripherals designed for penetration testing, cybersecurity training, and red team operations. They appear as normal devices, adapters, data blockers, or cables but contain hidden electronics capable of executing bidirectional HID attacks and are capable of being controlled locally or remotely (via C2)

# What is Red Teaming

Red teaming in cybersecurity is the practice of simulating real-world cyberattacks to test an organization’s defenses. A red team acts as an adversary, attempting to breach systems, applications, networks, or even physical security to uncover vulnerabilities and blind spots that could be exploited by real adversaries.

# Why Red Teaming

Uncovers hidden vulnerabilities across systems, people, and physical environments

Tests the resilience of incident response—revealing gaps in detection, communication, and recovery

Enhances security culture within organizations and real-world simulations boost employee awareness

Supports compliance and risk management, showing regulators you're rigorously evaluating threats

Prioritizes mitigation efforts, focusing on vulnerabilities most likely exploited by attackers.

And sometimes all of this involves going undercover and planting devices….

# Movie vs Real World and The Truth of "Hacking"

# Movie vs Real World and The Truth of "Hacking"

Ok still not quite it...
But the magic button that hacks doesn't exist
Real world "hacking" requires:
Carefully designing and crafting to a target
AKA Very Boring **WHATS GOING ON HERE?**

# The Tools of the Trade

Just a few …

# Identifying Targets

- Be Goal Oriented:
    - Much of the work takes place outside of actually running the operation.
    - Sometimes you will be let in, so you have to blend in and find a place to connect
    - Sometimes you will need to find reasons for people to use your devices
- Get In and Active:
    - Conduct site visits
        - Access Controls
        - Places to Plant Devices and Resources
        - How to Connect "out"
    - Design pretexts tailored to the facility and target
        - What systems do you want access to?

# Access Granted

You got access! Great job! Now what?

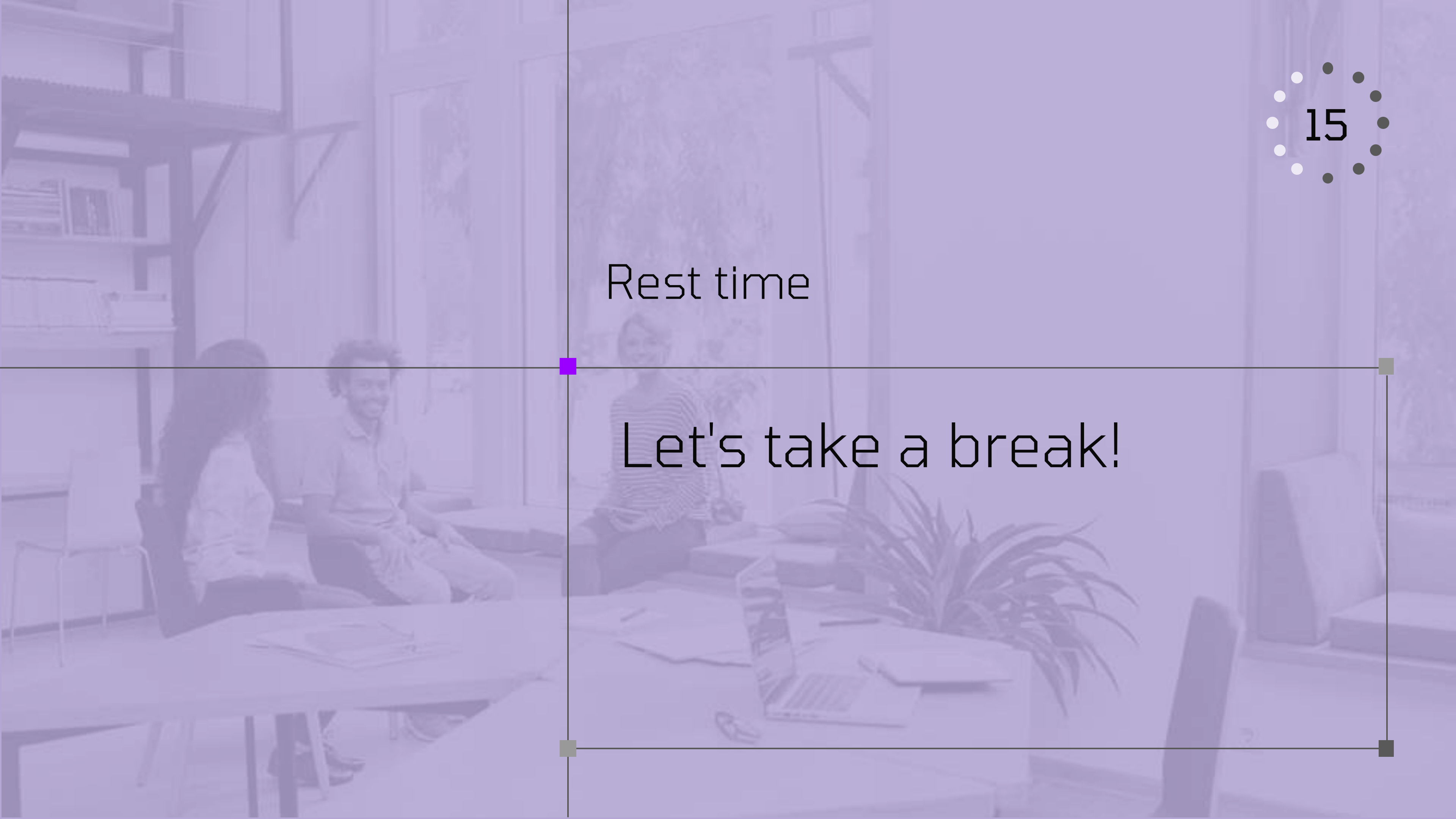
Got your bag of tricks ready? Right

Of course you do

# You have physical access… now what?

Ask yourself some questions:

- Do you have physical access and can you remain nearby?
- Avoid common commands, scripts, actions that would give you away
- Make your device look like the devices expected in the environment
- Setup situation you can see if the device was used
    - Bonus: In v4 you can monitor CAPS! ;)



## Rest time

# Let's take a break!

00

# O.MG * Basics

# O.MG (Plug) Elite Basics

- Hopefully you have already setup your OMG Device
- If not now is a good time to make sure your drivers are loaded

- https://o.mg.lol/setup/

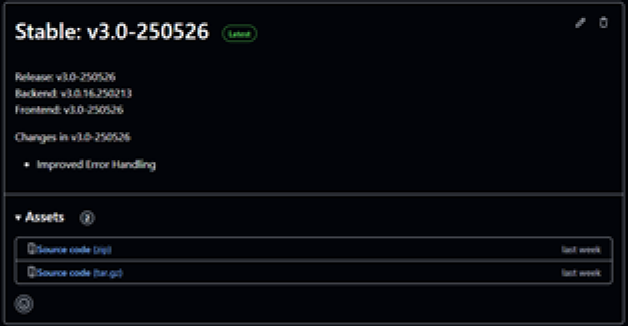
# O.MG (Plug) Basics

- Web UI
- Slots / Files
- Configurable Triggers and Visibility
- HIDX and C2
- DuckyScript 3.x

# The Hardware

# Firmware Flashing

## Python Flasher VS Web Flasher



Stable: v3.0-250526

Release: v3.0-250526
Backend: v3.0.16.250213
Frontend: v3.0-250526

Changes in v3.0-250526

- Improved Error Handling

## O.MG Web Flasher

○ **Step 1:** Plug-In

First, connect the O.MG Programmer to your computer. Then connect **the active end** of the O.MG Device into the programmer. Need help or more details? Click the Help button below.

○ **Step 2:** Connect

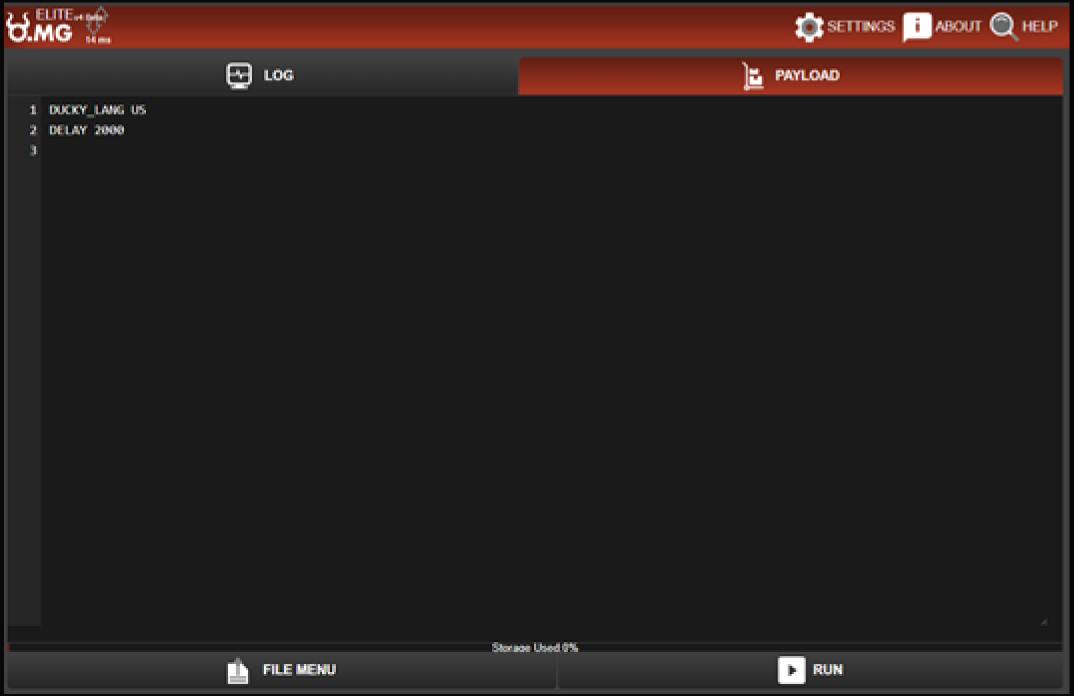
○ **Step 3:** Flash

# Final Chance!

**Optionally Connect to WiFi:**
SSID: HIDLAB
Pass: briskkey14
(Treat this network as hostile). :)

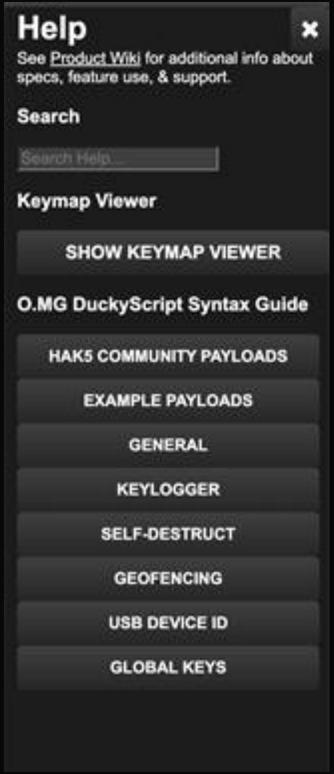
Files/Resources/ZIPs can be accessed at 192.168.1.10 or (files.lan)
Or download them from https://da.gd/mischief

# The Web Ui



```
DUCKY_LANG US
DELAY 2000
```

# Built In Help



## Help

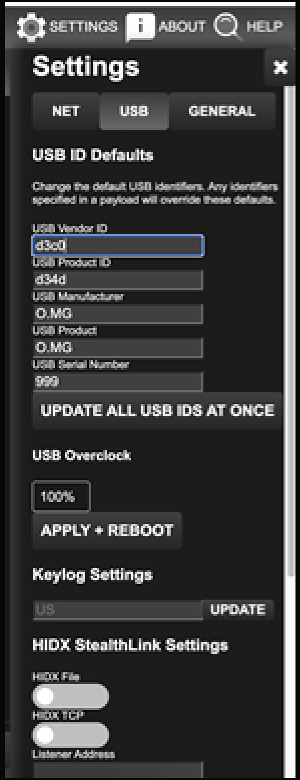
See Product Wiki for additional info about specs, feature use, & support.

**Search**

**Keymap Viewer**

**O.MG DuckyScript Syntax Guide**

# USB Settings



## Settings

### USB ID Defaults

Change the default USB identifiers. Any identifiers specified in a payload will override these defaults.

USB Vendor ID: d3c0

USB Product ID: d34d

USB Manufacturer: O.MG

USB Product: O.MG

USB Serial Number: 999

### USB Overclock

100%

### Keylog Settings

US

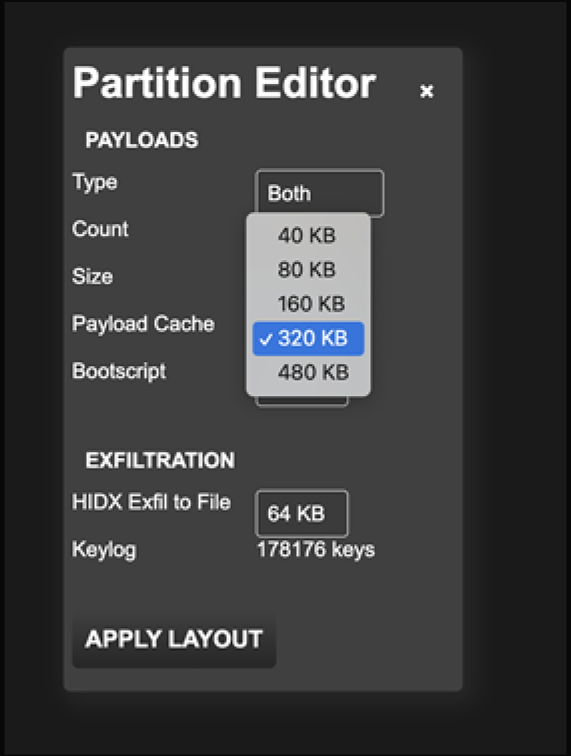
### HIDX StealthLink Settings

HIDX File

HIDX TCP

Listener Address

# Partition Editor



Partition Editor

PAYLOADS

Type: Both

Count

Size

Payload Cache

Bootscript

EXFILTRATION

HIDX Exfil to File: 64 KB

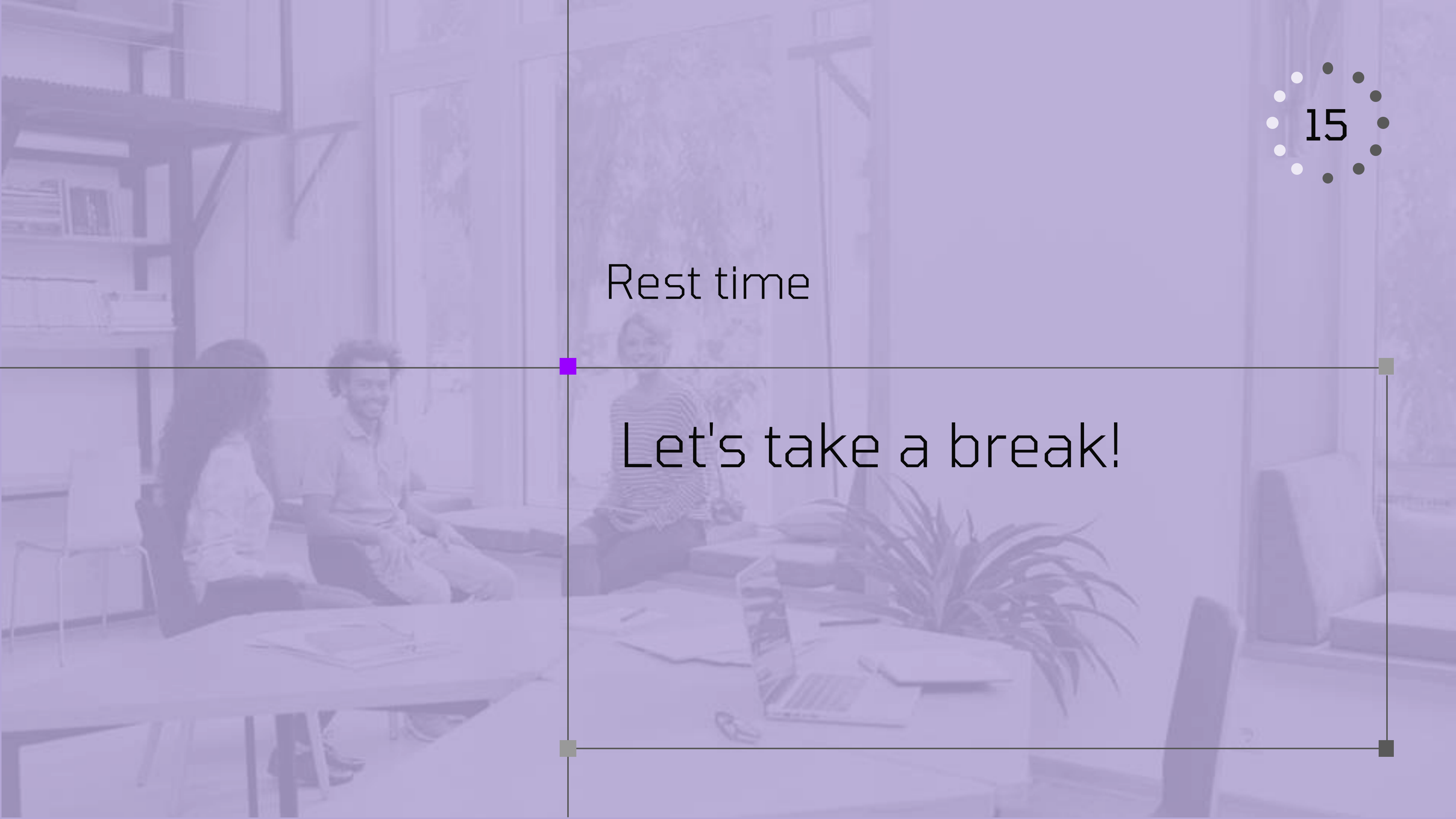
Keylog: 178176 keys

# Now that you've seen a bit...

*Pair up and attack each others computers... Kidding*

Please connect to your own device and start configuring things!

We assume you will be using the shared network.
Make sure your O.MG Devices are uniquely named.



15

## Rest time

# Let's take a break!

30

USB Primer and Understanding USB

A quick dive into some really complicated stuff

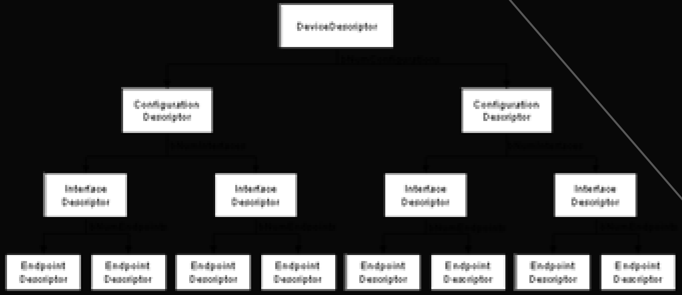
# Understanding USB

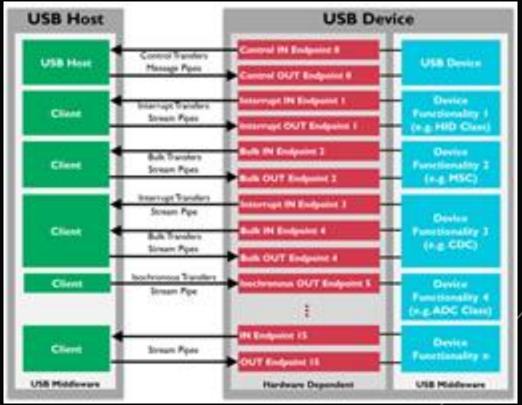
# History Lessons - USB

- Developed early 1990s and released in 1996(ish) by the USB Implementers Forum
    - - iMac G3 was the first widely sold computer to natively have USB
    - Microsoft, IBM, NEC, Compaq, DEC, and other companies part of working group
- Developed out of necessity - Many different ports and standard often proprietary
    - Serial, Parallel, Game, PS2, and other ports
    - Remember Apple Talk?
- We are on the 4th implementation
    - USB 1.0, 1.1, 2.0, 3.0, 3.1, 3.2, and now USB4 (USB 4 2.0)

# USB Basics

- Host Controller → Root Hub → Hub(s) → Device(s)
- USB Speeds (modes) have different timing requirements and therefore cannot be used interchangeably by the same device.
- USB Modes of Operation
  - Low Speed (LS) - 1.5 Mbps
    - Primarily used for Human Interface Devices
  - Full Speed (FS) - 12Mbps
    - “baseline” speed for many devices - Audio, Scanners, POS Systems, Modems, Keyboards, Headsets, etc
  - High Speed (HS) - 480Mbps
    - Storage Devices, etc. Different signaling then LS or FS
  - SuperSpeed (SS) - 10Gbps (USB 3.1) or 20Gbps (USB 3.2)
    - Uses different encoding (8b/10b to 128b/132b) and this is where naming gets complicated
    - Large Storage, High Bandwidth Devices
- After USB 3.0, Naming Goes Out the Window:
  - Gen 1x1
  - Gen 2x1
  - Gen 2x1
  - Gen 3x2
  - And USB 4 2.0



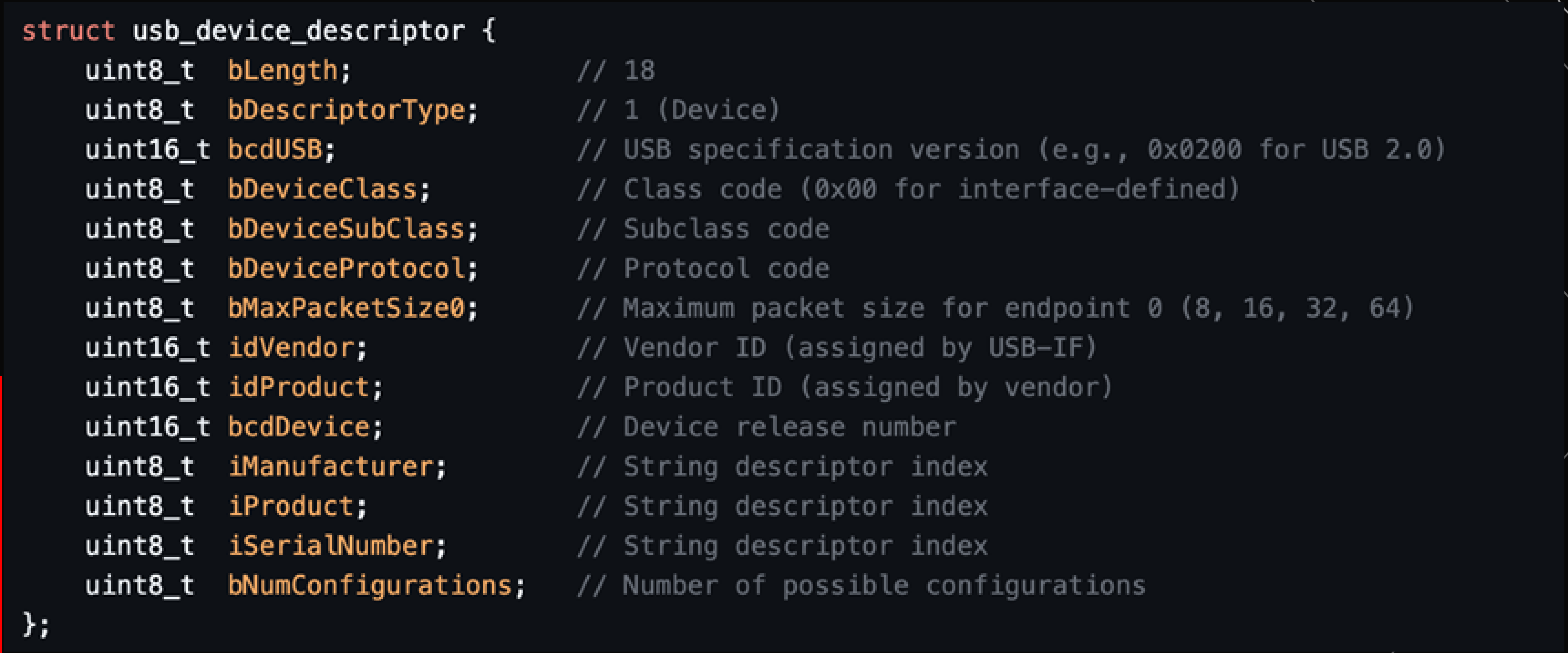


# USB Negotiation

- USB relies on descriptors, which describe to the host everything the device can and cannot do:
    - Device Descriptors - All Information about USB Device Present
    - Configuration Descriptors - One or more, describes power controls and interfaces
        - Interface / Endpoint Descriptors - Additional Endpoints presented by the device
- Negotiation Occurs in Stages:
    - Early: Connection Sensing and Speed Negotiation
    - Get Device Descriptor
    - Get / Set Address
    - Get Configuration Descriptor(s) (e.g. Interface, Endpoint, etc)
    - Get String Descriptors (Human Readable) if applicable
- Descriptors are hierarchical:

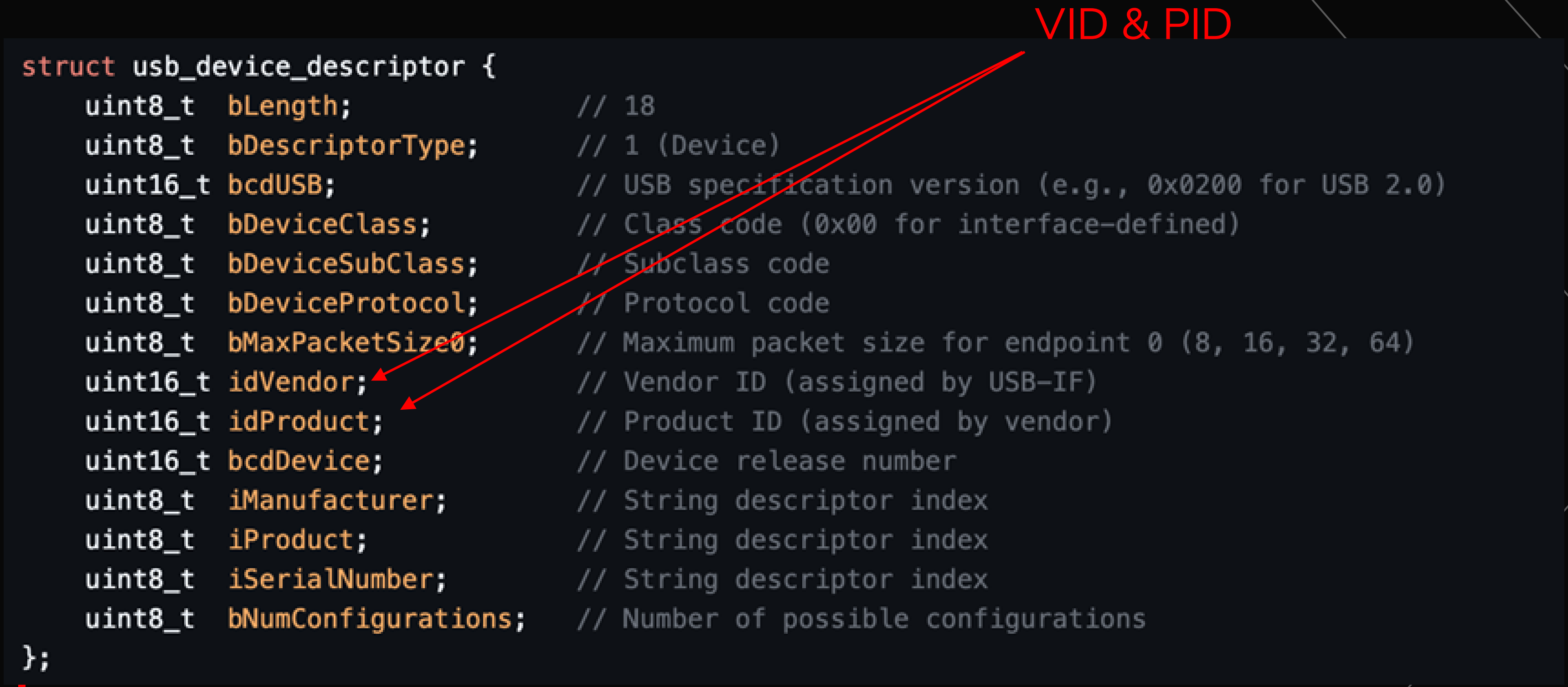
    - Device → Configuration(s) → Interface(s) → Endpoint(s)

# USB Descriptors and Negotiation



```
struct usb_device_descriptor {
    uint8_t  bLength;              // 18
    uint8_t  bDescriptorType;      // 1 (Device)
    uint16_t bcdUSB;               // USB specification version (e.g., 0x0200 for USB 2.0)
    uint8_t  bDeviceClass;         // Class code (0x00 for interface-defined)
    uint8_t  bDeviceSubClass;      // Subclass code
    uint8_t  bDeviceProtocol;      // Protocol code
    uint8_t  bMaxPacketSize0;      // Maximum packet size for endpoint 0 (8, 16, 32, 64)
    uint16_t idVendor;             // Vendor ID (assigned by USB-IF)
    uint16_t idProduct;            // Product ID (assigned by vendor)
    uint16_t bcdDevice;            // Device release number
    uint8_t  iManufacturer;        // String descriptor index
    uint8_t  iProduct;             // String descriptor index
    uint8_t  iSerialNumber;        // String descriptor index
    uint8_t  bNumConfigurations;   // Number of possible configurations
};
```

# USB Descriptors and Negotiation



```
struct usb_device_descriptor {
    uint8_t  bLength;            // 18
    uint8_t  bDescriptorType;    // 1 (Device)
    uint16_t bcdUSB;             // USB specification version (e.g., 0x0200 for USB 2.0)
    uint8_t  bDeviceClass;       // Class code (0x00 for interface-defined)
    uint8_t  bDeviceSubClass;    // Subclass code
    uint8_t  bDeviceProtocol;    // Protocol code
    uint8_t  bMaxPacketSize0;    // Maximum packet size for endpoint 0 (8, 16, 32, 64)
    uint16_t idVendor;           // Vendor ID (assigned by USB-IF)
    uint16_t idProduct;          // Product ID (assigned by vendor)
    uint16_t bcdDevice;          // Device release number
    uint8_t  iManufacturer;      // String descriptor index
    uint8_t  iProduct;           // String descriptor index
    uint8_t  iSerialNumber;      // String descriptor index
    uint8_t  bNumConfigurations; // Number of possible configurations
};
```

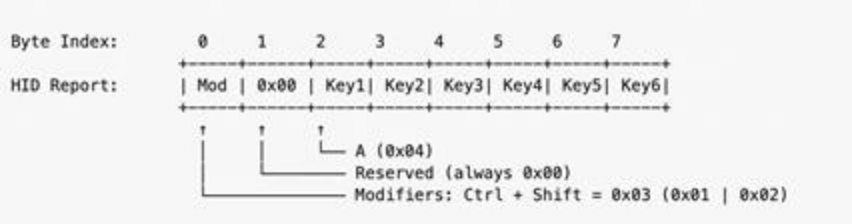
VID & PID

# USB HID

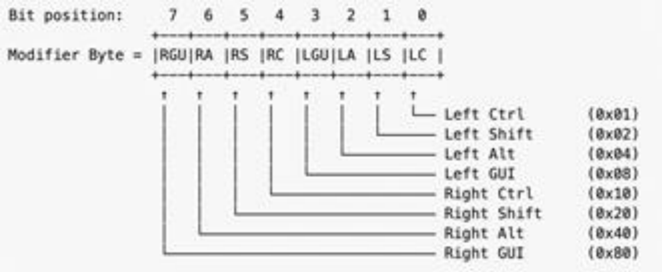
- HID Class Definition: Defines how human interface devices (keyboards, mice, etc.) communicate over USB.
- HID Reports: Structured data packets described by Report Descriptors.
- Usage Pages & IDs: Groupings of related controls (e.g., Generic Desktop, Keyboard).
- Descriptor Structure:
  - Report Descriptor: Sequence of items (Main, Global, Local) that describe report format.
  - Collection: Logical grouping (e.g., Application for keyboard).

```
0x05, 0x01,         // Usage Page (Generic Desktop)
0x09, 0x06,         // Usage (Keyboard)
0xA1, 0x01,         // Collection (Application)
  0x05, 0x07,       //   Usage Page (Keyboard)
  0x19, 0xE0,       //   Usage Minimum (Left Ctrl)
  0x29, 0xE7,       //   Usage Maximum (Right GUI)
  0x15, 0x00,       //   Logical Minimum (0)
  0x25, 0x01,       //   Logical Maximum (1)
  0x75, 0x01,       //   Report Size (1)
  0x95, 0x08,       //   Report Count (8)
  0x81, 0x02,       //   Input (Data,Var,Abs) ; Modifier byte
  0x75, 0x08,       //   Report Size (8)
  0x95, 0x01,       //   Report Count (1)
  0x81, 0x03,       //   Input (Const,Var,Abs) ; Reserved
  0x75, 0x08,       //   Report Size (8)
  0x95, 0x06,       //   Report Count (6)
  0x15, 0x00,       //   Logical Minimum (0)
  0x25, 0x65,       //   Logical Maximum (101)
  0x05, 0x07,       //   Usage Page (Keyboard)
  0x19, 0x00,       //   Usage Minimum (Reserved)
  0x29, 0x65,       //   Usage Maximum (Keyboard Application)
  0x81, 0x00,       //   Input (Data,Arr,Abs) ; Keycodes array
0xC0                // End Collection
```

# Sending a Report



```
Byte Index:        0     1      2     3     4     5     6     7
                +-----+------+-----+-----+-----+-----+-----+-----+
HID Report:     | Mod | 0x00 | Key1| Key2| Key3| Key4| Key5| Key6|
                +-----+------+-----+-----+-----+-----+-----+-----+
                   ↑     ↑      ↑
                   |     |      └── A (0x04)
                   |     └───────── Reserved (always 0x00)
                   └─────────────── Modifiers: Ctrl + Shift = 0x03 (0x01 | 0x02)
```



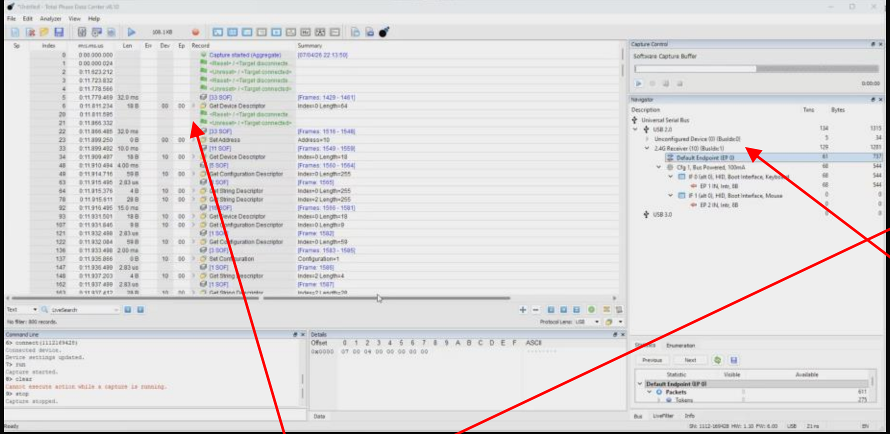
```
Bit position:      7   6   5   4   3   2   1   0
                 +---+---+---+---+---+---+---+---+
Modifier Byte =  |RGU|RA |RS |RC |LGU|LA |LS |LC |
                 +---+---+---+---+---+---+---+---+
                   ↑   ↑   ↑   ↑   ↑   ↑   ↑   ↑
                   |   |   |   |   |   |   |   └── Left Ctrl       (0x01)
                   |   |   |   |   |   |   └────── Left Shift      (0x02)
                   |   |   |   |   |   └────────── Left Alt        (0x04)
                   |   |   |   |   └────────────── Left GUI        (0x08)
                   |   |   |   └────────────────── Right Ctrl      (0x10)
                   |   |   └────────────────────── Right Shift     (0x20)
                   |   └────────────────────────── Right Alt       (0x40)
                   └────────────────────────────── Right GUI       (0x80)
```

- HID Reports: 8 bytes
  0x02, 0x00, 0x1C, 0x00, 0x00, 0x00, 0x00, 0x00 = Send “A”
- Multiple keys can be pressed, as a key is released, remaining keys shifted on report

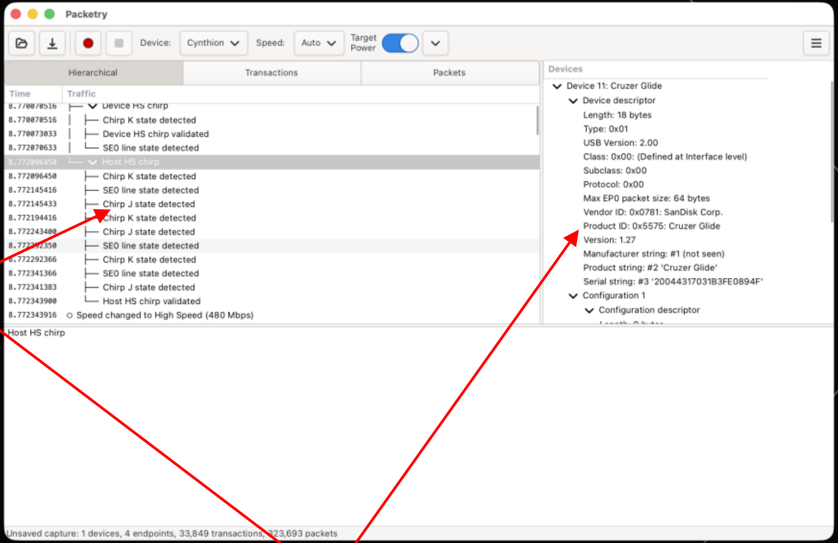
# USB Analyzers

- USB is complex and analysis must take place across the entire process including handshake.
- To capture all the layers (including what might not be visible to Wireshark) you need specialized hardware
- Some "cheaper" variants but software / firmware support is a major factor

# Capture Tooling



USB Traffic



```
Time          Traffic
8.770070516   ├── ∨ Device HS chirp
8.770070516   │   ├── Chirp K state detected
8.770073033   │   ├── Device HS chirp validated
8.772070633   │   └── SE0 line state detected
8.772096450   └── ∨ Host HS chirp
8.772096450       ├── Chirp K state detected
8.772145416       ├── SE0 line state detected
8.772145433       ├── Chirp J state detected
8.772194416       ├── Chirp K state detected
8.772243400       ├── Chirp J state detected
8.772292350       ├── SE0 line state detected
8.772292366       ├── Chirp K state detected
8.772341366       ├── SE0 line state detected
8.772341383       ├── Chirp J state detected
8.772343900       └── Host HS chirp validated
8.772343916   ○ Speed changed to High Speed (480 Mbps)
```

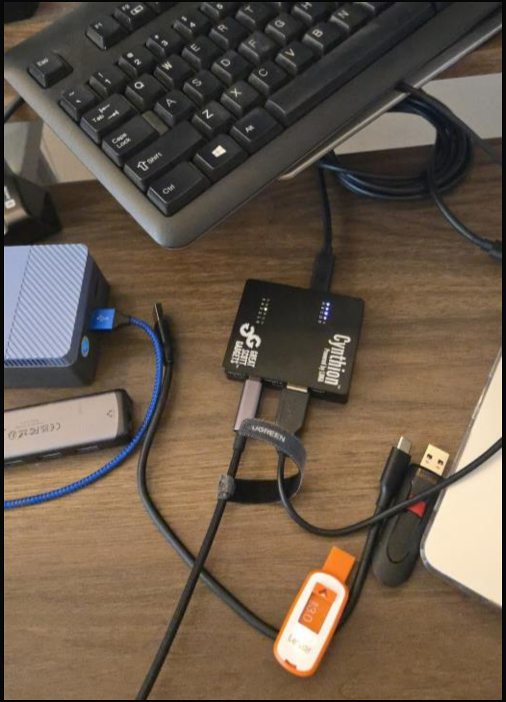
Host HS chirp

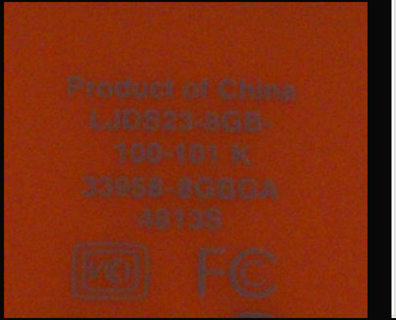
```
∨ Device 11: Cruzer Glide
  ∨ Device descriptor
     Length: 18 bytes
     Type: 0x01
     USB Version: 2.00
     Class: 0x00: (Defined at Interface level)
     Subclass: 0x00
     Protocol: 0x00
     Max EP0 packet size: 64 bytes
     Vendor ID: 0x0781: SanDisk Corp.
     Product ID: 0x5575: Cruzer Glide
     Version: 1.27
     Manufacturer string: #1 (not seen)
     Product string: #2 'Cruzer Glide'
     Serial string: #3 '20044317031B3FE0894F'
  ∨ Configuration 1
    ∨ Configuration descriptor
```

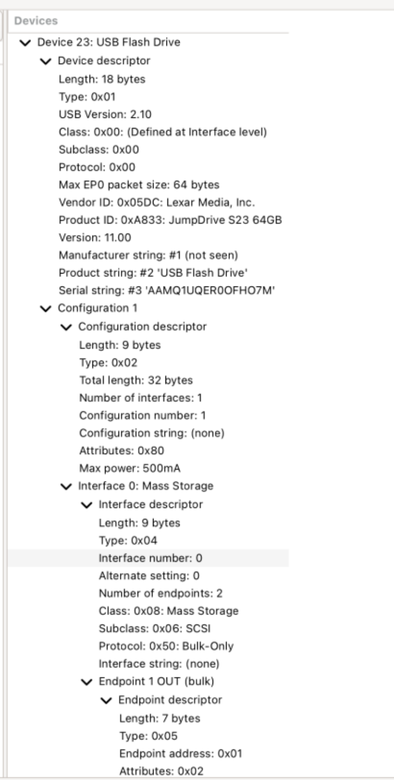
Unsaved capture: 1 devices, 4 endpoints, 33,849 transactions, 323,693 packets

Device [Descriptors]

# Fun with Analysis







```
Devices
⌄ Device 23: USB Flash Drive
  ⌄ Device descriptor
      Length: 18 bytes
      Type: 0x01
      USB Version: 2.10
      Class: 0x00: (Defined at Interface level)
      Subclass: 0x00
      Protocol: 0x00
      Max EP0 packet size: 64 bytes
      Vendor ID: 0x05DC: Lexar Media, Inc.
      Product ID: 0xA833: JumpDrive S23 64GB
      Version: 11.00
      Manufacturer string: #1 (not seen)
      Product string: #2 'USB Flash Drive'
      Serial string: #3 'AAMQ1UQER0OFHO7M'
  ⌄ Configuration 1
    ⌄ Configuration descriptor
        Length: 9 bytes
        Type: 0x02
        Total length: 32 bytes
        Number of interfaces: 1
        Configuration number: 1
        Configuration string: (none)
        Attributes: 0x80
        Max power: 500mA
    ⌄ Interface 0: Mass Storage
      ⌄ Interface descriptor
          Length: 9 bytes
          Type: 0x04
          Interface number: 0
          Alternate setting: 0
          Number of endpoints: 2
          Class: 0x08: Mass Storage
          Subclass: 0x06: SCSI
          Protocol: 0x50: Bulk-Only
          Interface string: (none)
      ⌄ Endpoint 1 OUT (bulk)
        ⌄ Endpoint descriptor
            Length: 7 bytes
            Type: 0x05
            Endpoint address: 0x01
            Attributes: 0x02
```

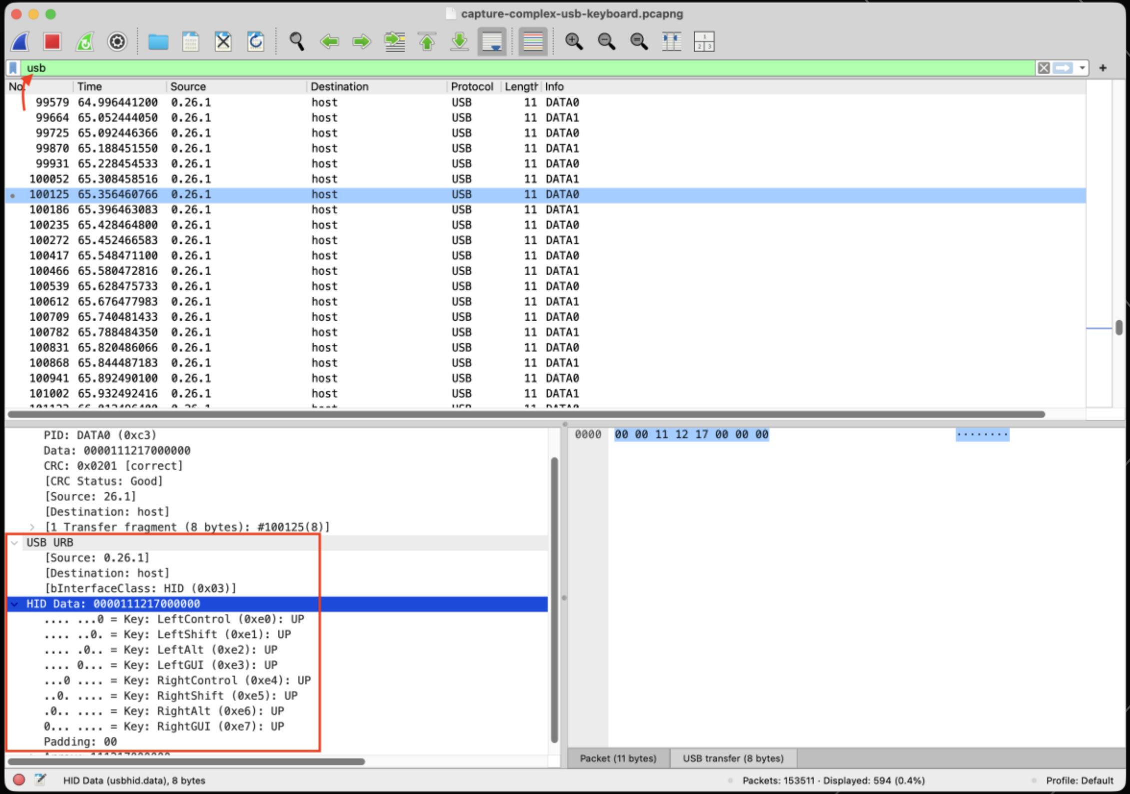
# When Analysis Fails

| Time | Traffic |
|---|---|
| 0.000000000 | ○ Capture started at Low Speed (1.5 Mbps) |
| 0.000118866 | ○ › 237 Low Speed keepalive groups |
| 0.000127500 | › Polling 236 times for unidentified transfer on endpoint 24.1 IN |
| 0.000167466 | › Polling 236 times for unidentified transfer on endpoint 24.2 IN |
| 1.881540966 | ○ Bus reset |
| 2.460332283 | › 1 invalid groups |
| 2.465366200 | › 1 invalid groups |
| 2.470334116 | › 1 invalid groups |
| 2.471256766 | › 1 invalid groups |
| 2.473316066 | › 1 invalid groups |
| 2.475256700 | › 1 invalid groups |
| 2.476272683 | › 1 invalid groups |
| 2.477268000 | › 1 invalid groups |
| 2.478253316 | › 1 invalid groups |
| 2.480253950 | › 1 invalid groups |

- USB Modes (Low, Full, High Speed) are negotiated at start
- Mid-stream capture generally but does not always work
- At the wrong speed or monitoring the wrong endpoint you can miss information

# Back to Wireshark

- While OS capture limitations still exist, Wireshark is perfectly capable of analyzing exported captures
- What is being said here?

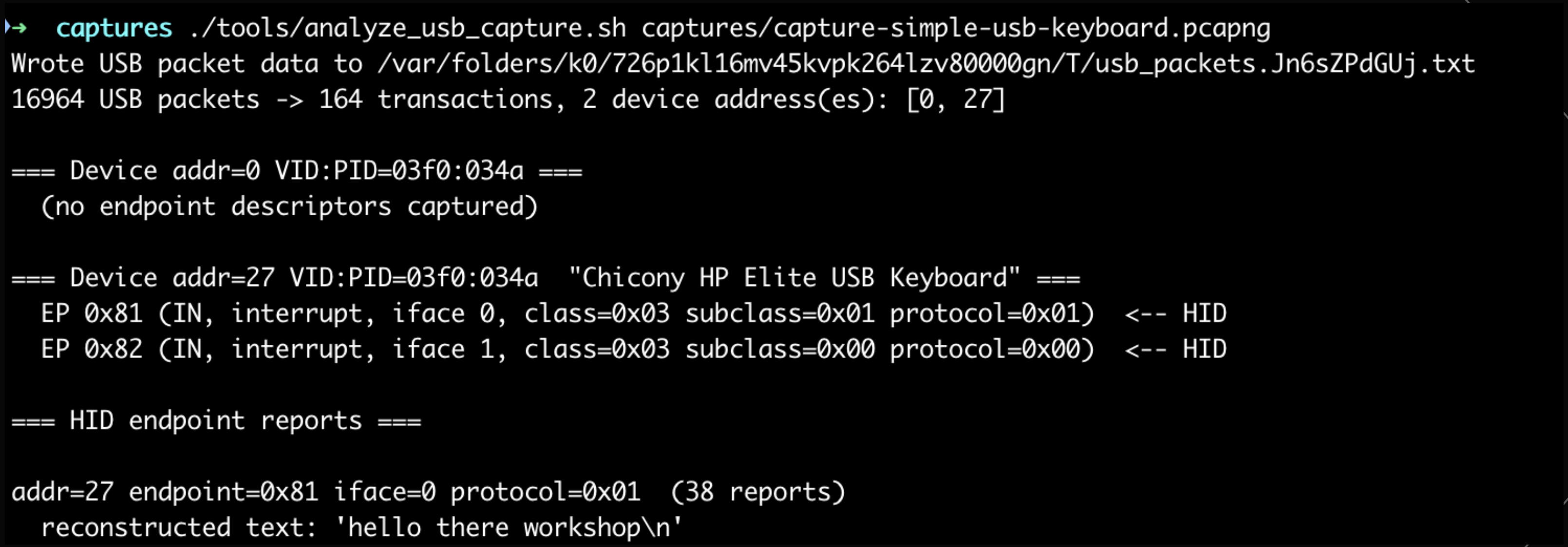


```
No.     Time          Source  Destination  Protocol  Length  Info
 99579  64.996441200  0.26.1  host         USB       11      DATA0
 99664  65.052444050  0.26.1  host         USB       11      DATA1
 99725  65.092446366  0.26.1  host         USB       11      DATA0
 99870  65.188451550  0.26.1  host         USB       11      DATA1
 99931  65.228454533  0.26.1  host         USB       11      DATA0
100052  65.308458516  0.26.1  host         USB       11      DATA1
100125  65.356460766  0.26.1  host         USB       11      DATA0
100186  65.396463083  0.26.1  host         USB       11      DATA1
100235  65.428464800  0.26.1  host         USB       11      DATA0
100272  65.452466583  0.26.1  host         USB       11      DATA1
100417  65.548471100  0.26.1  host         USB       11      DATA0
100466  65.580472816  0.26.1  host         USB       11      DATA1
100539  65.628475733  0.26.1  host         USB       11      DATA0
100612  65.676477983  0.26.1  host         USB       11      DATA1
100709  65.740481433  0.26.1  host         USB       11      DATA0
100782  65.788484350  0.26.1  host         USB       11      DATA1
100831  65.820486066  0.26.1  host         USB       11      DATA0
100868  65.844487183  0.26.1  host         USB       11      DATA1
100941  65.892490100  0.26.1  host         USB       11      DATA0
101002  65.932492416  0.26.1  host         USB       11      DATA1
```

```
    PID: DATA0 (0xc3)
    Data: 0000111217000000
    CRC: 0x0201 [correct]
    [CRC Status: Good]
    [Source: 26.1]
    [Destination: host]
  > [1 Transfer fragment (8 bytes): #100125(8)]
v USB URB
    [Source: 0.26.1]
    [Destination: host]
    [bInterfaceClass: HID (0x03)]
v HID Data: 0000111217000000
    .... ...0 = Key: LeftControl (0xe0): UP
    .... ..0. = Key: LeftShift (0xe1): UP
    .... .0.. = Key: LeftAlt (0xe2): UP
    .... 0... = Key: LeftGUI (0xe3): UP
    ...0 .... = Key: RightControl (0xe4): UP
    ..0. .... = Key: RightShift (0xe5): UP
    .0.. .... = Key: RightAlt (0xe6): UP
    0... .... = Key: RightGUI (0xe7): UP
    Padding: 00
```

HID Data (usbhid.data), 8 bytes

# USB Decoder Tool

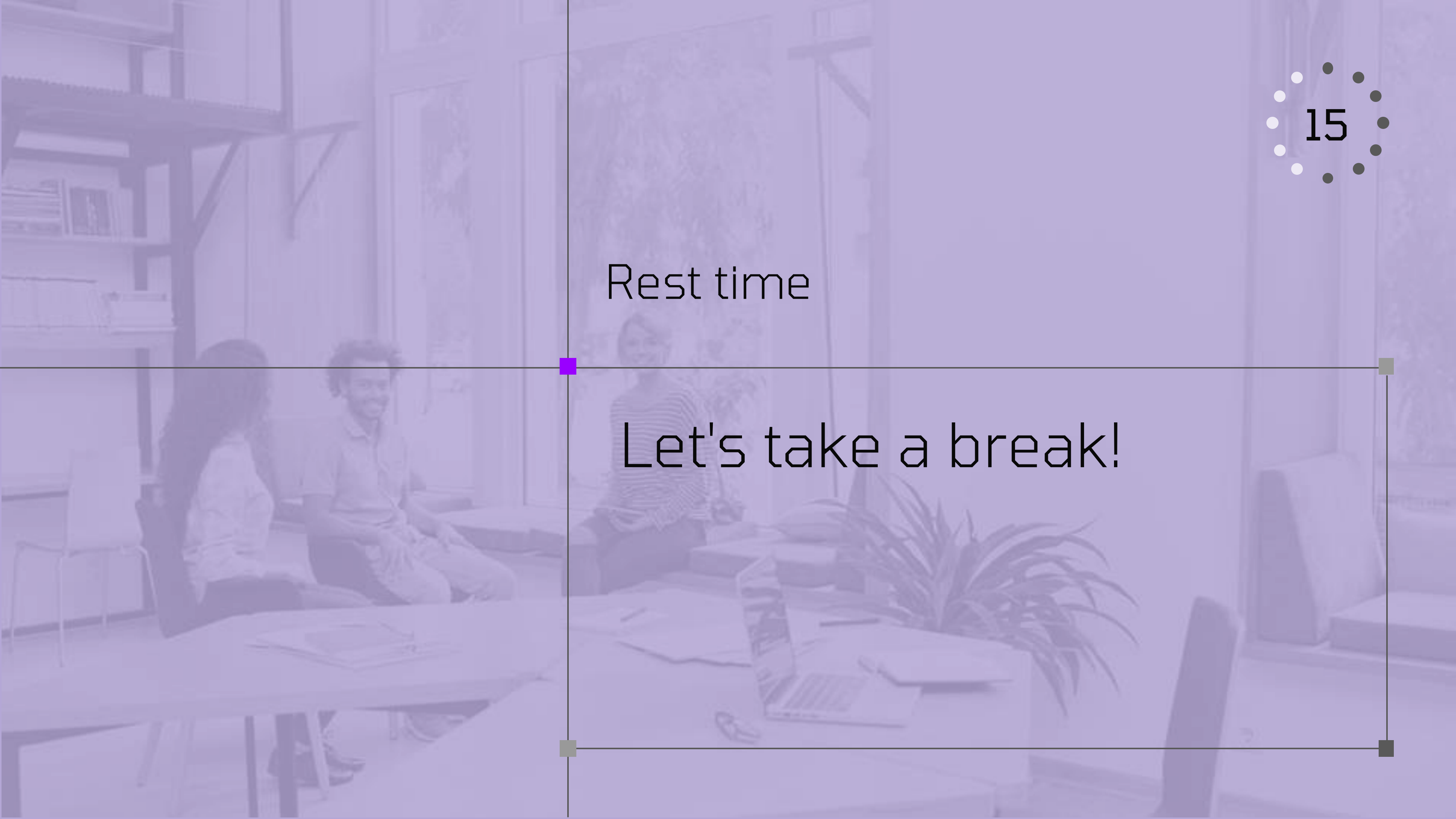


```
➜  captures ./tools/analyze_usb_capture.sh captures/capture-simple-usb-keyboard.pcapng
Wrote USB packet data to /var/folders/k0/726p1kl16mv45kvpk264lzv80000gn/T/usb_packets.Jn6sZPdGUj.txt
16964 USB packets -> 164 transactions, 2 device address(es): [0, 27]

=== Device addr=0 VID:PID=03f0:034a ===
  (no endpoint descriptors captured)

=== Device addr=27 VID:PID=03f0:034a  "Chicony HP Elite USB Keyboard" ===
  EP 0x81 (IN, interrupt, iface 0, class=0x03 subclass=0x01 protocol=0x01)  <-- HID
  EP 0x82 (IN, interrupt, iface 1, class=0x03 subclass=0x00 protocol=0x00)  <-- HID

=== HID endpoint reports ===

addr=27 endpoint=0x81 iface=0 protocol=0x01  (38 reports)
  reconstructed text: 'hello there workshop\n'
```

But for now, try to decode this by hand!

# Knowledge Break!

Let's look at all this through captures!



Rest time

# Let's take a break!

## 00

DuckyScript™ Basics

Quack! Quack! Quack!

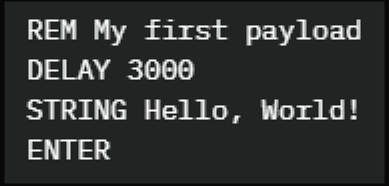
# DuckyScript™ Basics

# What's DuckyScript™?

DuckyScript™ is a scripting language initially created for the USB Rubber Ducky™, designed to be user friendly and easily readable. Later used for Hak5® HID attack gear and officially licensed devices such as O.MG devices

# Speaking like a Duck

What does DuckyScript™ look like?



```
REM My first payload
DELAY 3000
STRING Hello, World!
ENTER
```

# Speaking like a Duck

- Each line has a command and an optional parameter
- STRING(s) - Actual Input Text
    - STRING - MESSAGE
    - STRINGLN Message+New Line
- Actual Keys - i.e. ENTER, ALT
- Timings - i.e. DELAY
- Internal “Commands” - i.e. DEFINE
- Comments - REM

# Speaking like a Duck

- GUI Key - Meta Key - The “Windows Key”
- CUT, COPY, PASTE keys - Less common but useful for batch actions
- String GUI with other modifiers to access keyboard modifiers
- Group these tasks + STRING (typing) results in actions you can perform again and again
- Power of DuckyScript is making something easy to distribute and reproduce

# Variables & Function Blocks

```
FUNCTION greeting()
STRING Hello, World!
END_FUNCTION
```

```
FUNCTION greeetings_user(VAR1)
STRING Hello #VAR1, World!
END_FUNCTION
greetings_user(“Bob”)
```

# Blocks

```
REM_BLOCK
This is a big comment
And a few more lines
END_REM
```

# Delays

Delays ensure messages are sent not too quickly and to help simulate human activity or ensure timing

DELAY - Wait in MS

DEFAULT_DELAY - Delay before each command

DEFAULT_CHAR_DELAY - Delay between each character printed

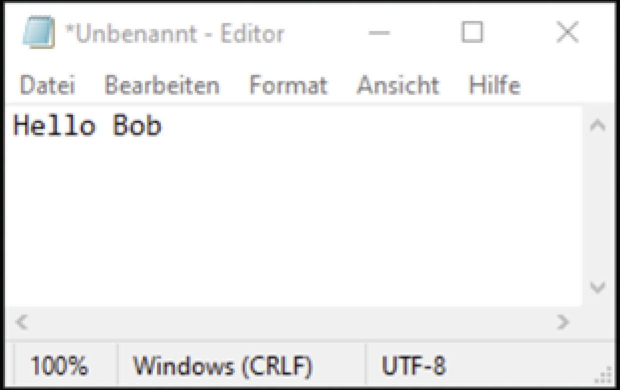
DEFAULT_DELAY_JITTER - Random

DEFAULT_DELAY between 0 and defined value

DEFAULT_CHAR_DELAY_JITTER - Random delay between DEFAULT_CHAR_DELAY and 0.

# DuckyScript™ Examples

```
DEFINE #FRIEND Bob
DELAY 2000
GUI r
DELAY 2000
STRING notepad
DELAY 500
ENTER
DELAY 2000
STRING Hello #FRIEND
```



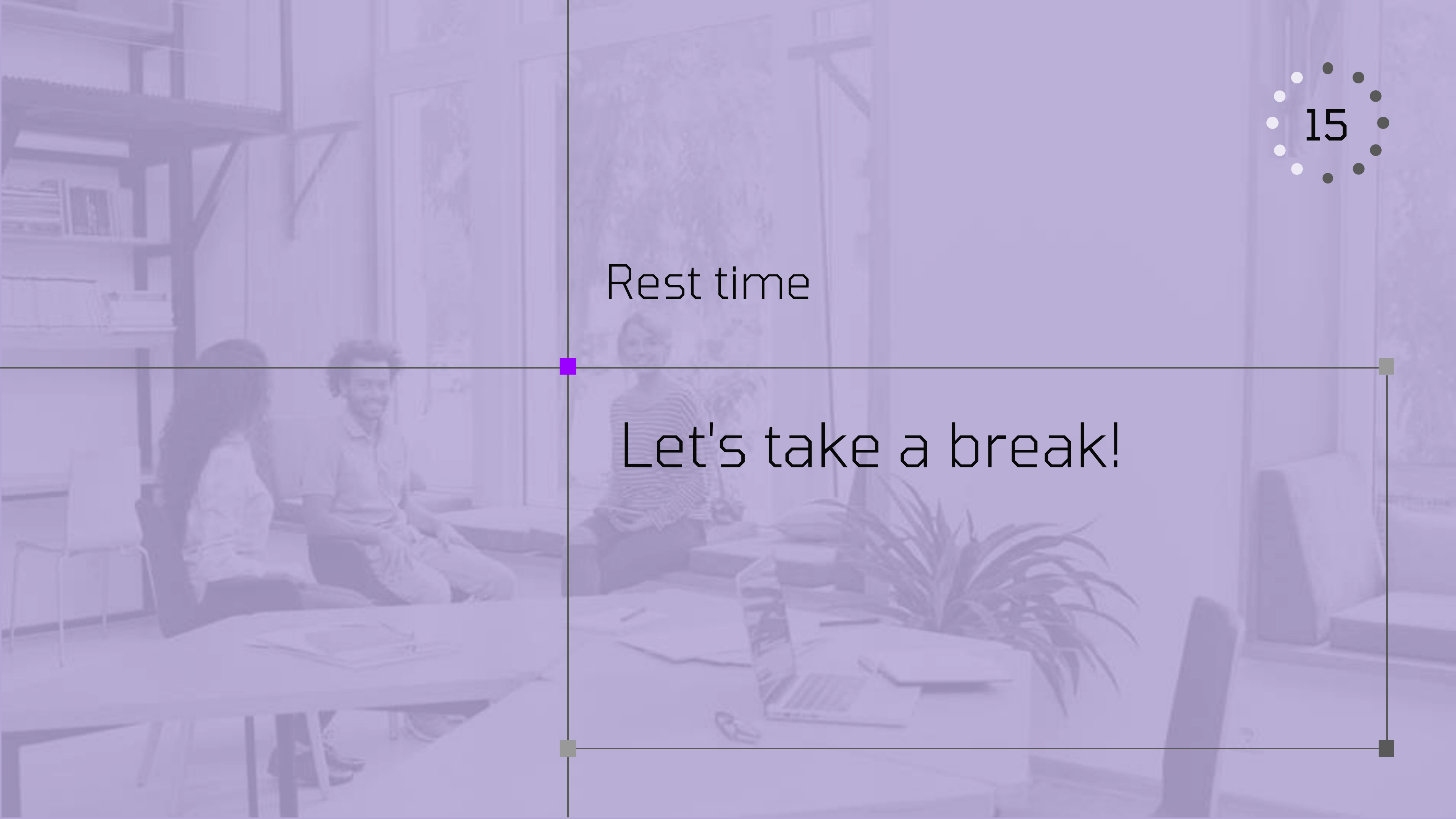
```
Hello Bob
```

# O.MG Specifics

- While O.MG Devices are compatible with DuckyScript 2.0 and 3.0 it has additional O.MG specific features
- Some additional capabilities
  - Control O.MG Hardware
  - Mouse Control
  - Geofencing
  - Triggers
  - Special Keys

# O.MG Specifics

- SELF-DESTRUCT ( and SELF-DESTRUCT-2): Erase all data and “brick” the device or let it run as a normal device
- KEYLOGGER - Toggle Keylogger! (Not applicable to Plugs)
- HIDX - Toggle HIDX
- IF_PRESENT (WAIT_FOR_PRESENT) - Scan for SSIDs and activate when found



15

Rest time

# Let's take a break!

00

Payloads

Recon, Setup, Weaponization, Debugging & Improving

# Payloads

# Planning Operations

1_

**Recon**

- Who to target
- How to deliver
- Target Operating System
- Target Language
- Realistic Goal

2_

**Setup**

- Adapt identifiers
- Plan Payload
- Setup Infrastructure
- Plan Access-Method

3_

**Weaponization**

- Create Payload

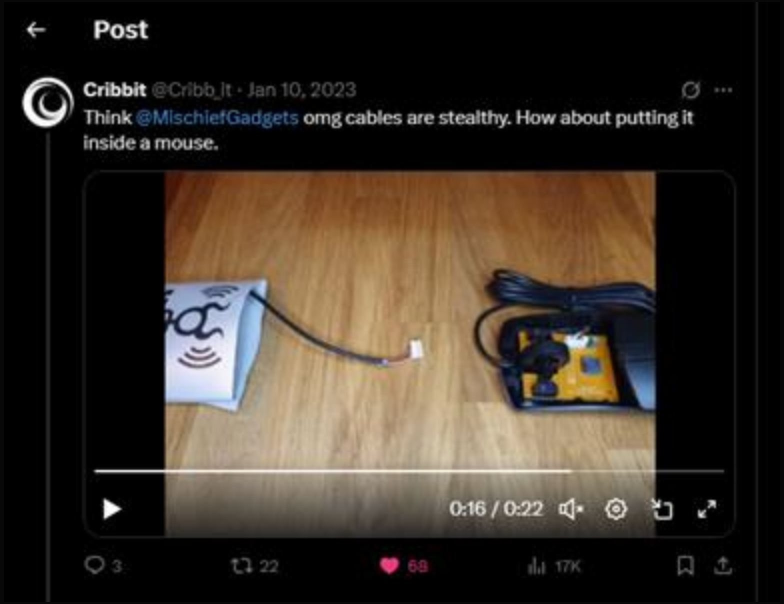
4_

**Debugging & Improving**

- Improve Timings
- Shorten Strings
- Guardrails
- Opsec

# 1. Recon

# 1. Recon



**Cribbit** @Cribb_It · Jan 10, 2023

Think @MischiefGadgets omg cables are stealthy. How about putting it inside a mouse.

# Planning Operations

1_

**Recon**

- Who to target
- How to deliver
- Target Operating System
- Target Language
- Realistic Goal

2_

**Setup**

- Adapt identifiers
- Plan Payload
- Setup Infrastructure
- Plan Access-Method

3_

**Weaponization**

- Create Payload

4_

**Debugging & Improving**

- Improve Timings
- Shorten Strings
- Guardrails
- Opsec

# 4. Setup

Preparation is the key to success (Alexander Graham Bell)

# Planning Operations

1_

**Recon**

- Who to target
- How to deliver
- Target Operating System
- Target Language
- Realistic Goal

2_

**Setup**

- Adapt identifiers
- Plan Payload
- Setup Infrastructure
- Plan Access-Method

3_

**Weaponization**

- Create Payload

4_

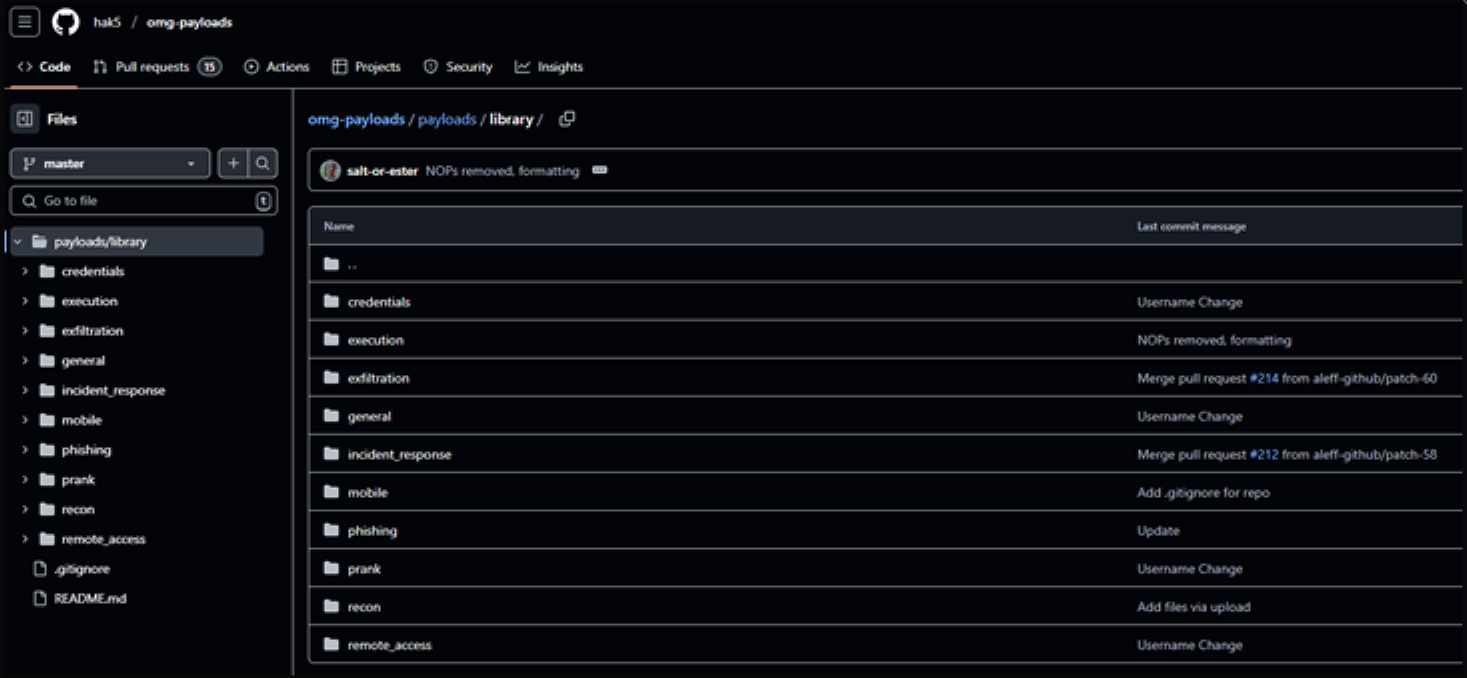
**Debugging & Improving**

- Improve Timings
- Shorten Strings
- Guardrails
- Opsec

# 4. Weaponization

- Creating payloads
- PowerShell in DuckyScript™
- Integrating internet hosted resources

# Public Payloads



omg-payloads / payloads / library /

salt-or-ester NOPs removed, formatting

| Name | Last commit message |
|---|---|
| .. | |
| credentials | Username Change |
| execution | NOPs removed, formatting |
| exfiltration | Merge pull request #214 from aleff-github/patch-60 |
| general | Username Change |
| incident_response | Merge pull request #212 from aleff-github/patch-58 |
| mobile | Add .gitignore for repo |
| phishing | Update |
| prank | Username Change |
| recon | Add files via upload |
| remote_access | Username Change |

# Hat on a Hat: PowerShell in DuckyScript™

# Planning Operations

1_

**Recon**

- Who to target
- How to deliver
- Target Operating System
- Target Language
- Realistic Goal

2_

**Setup**

- Adapt identifiers
- Plan Payload
- Setup Infrastructure
- Plan Access-Method

3_

**Weaponization**

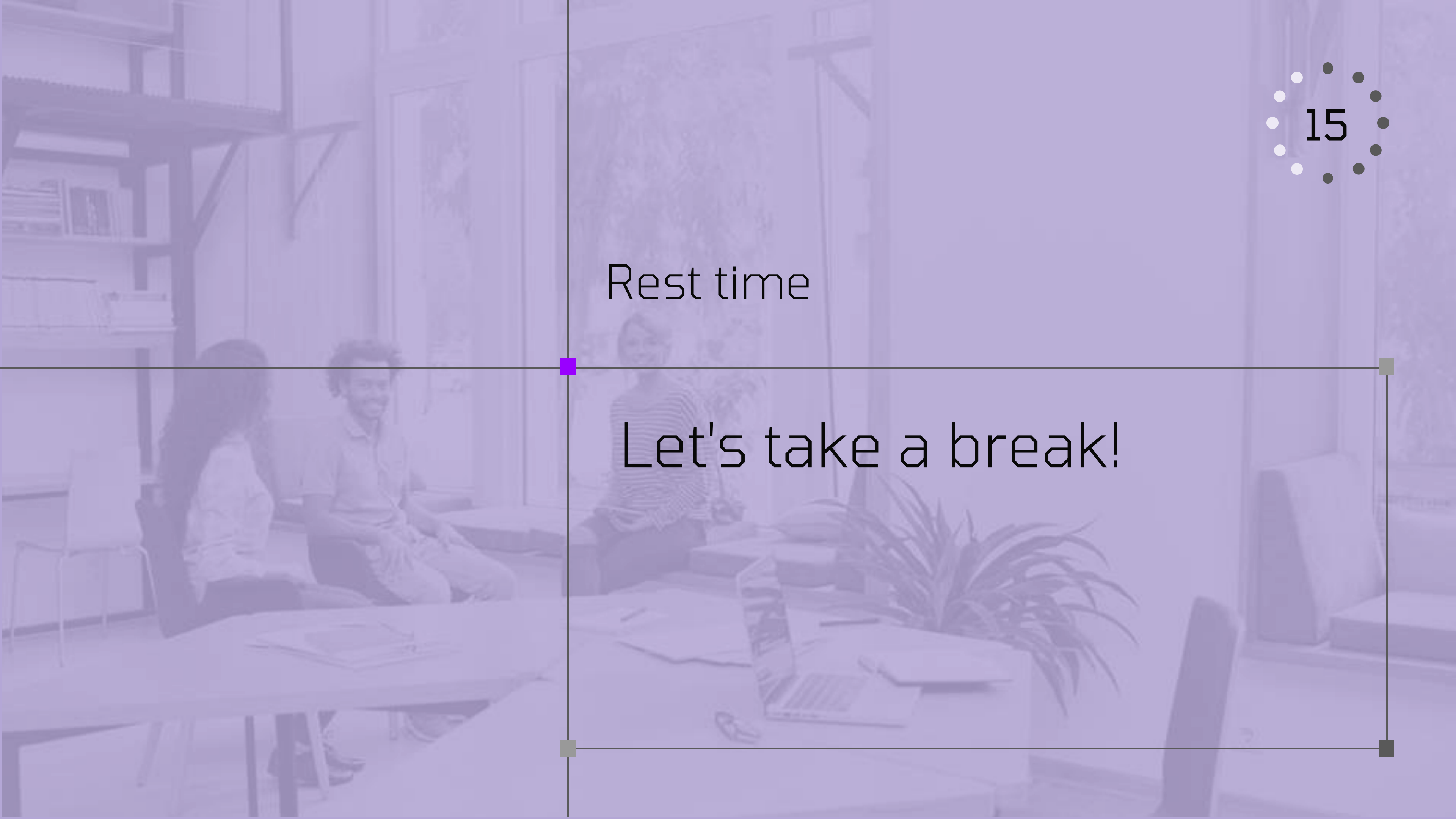
- Create Payload

4_

**Debugging & Improving**

- Improve Timings
- Shorten Strings
- Guardrails
- Opsec

# 4. Debugging & Improving



15

## Rest time

# Let's take a break!

00

Using Advanced Features

# O.MG Advanced Features and Demos

# HIDX Stealth Link

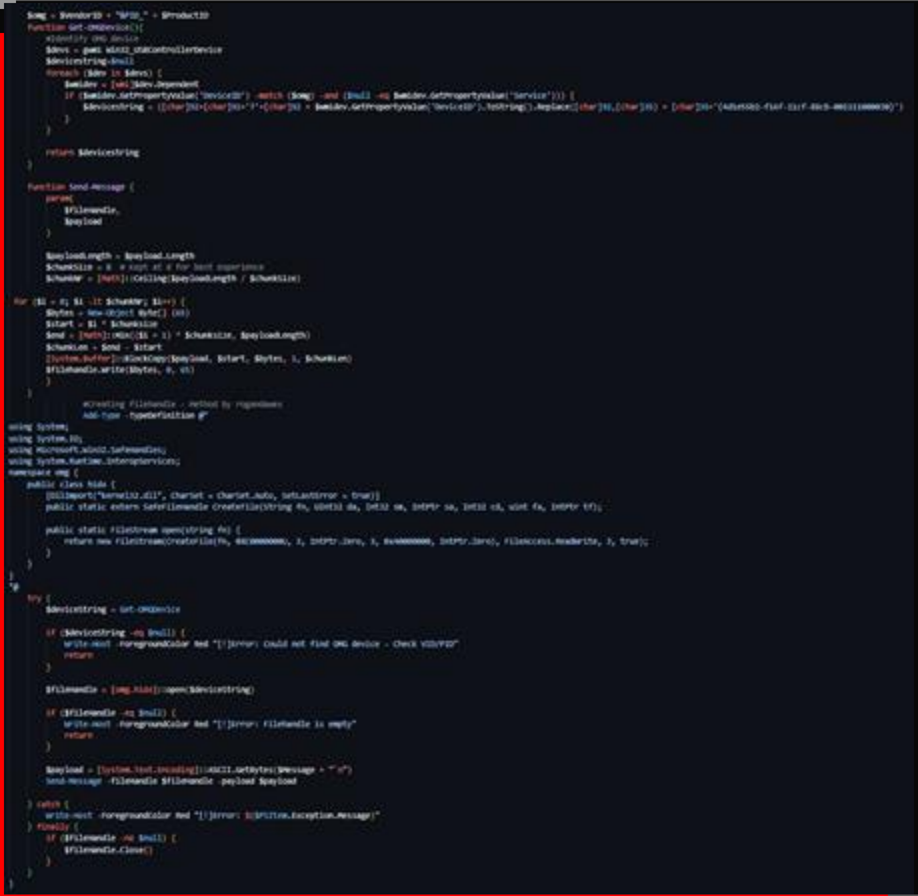
Jumping the Air Gap

# What is HIDX Stealth Link?

HIDX Stealth Link is a method where data is transferred using a covert channel.

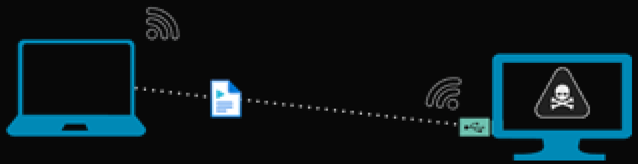
HID is used to establish a (bi)directional connection between the USB-Host and the O.MG device.

This creates the ability to exfiltrate data or the establish remote access.

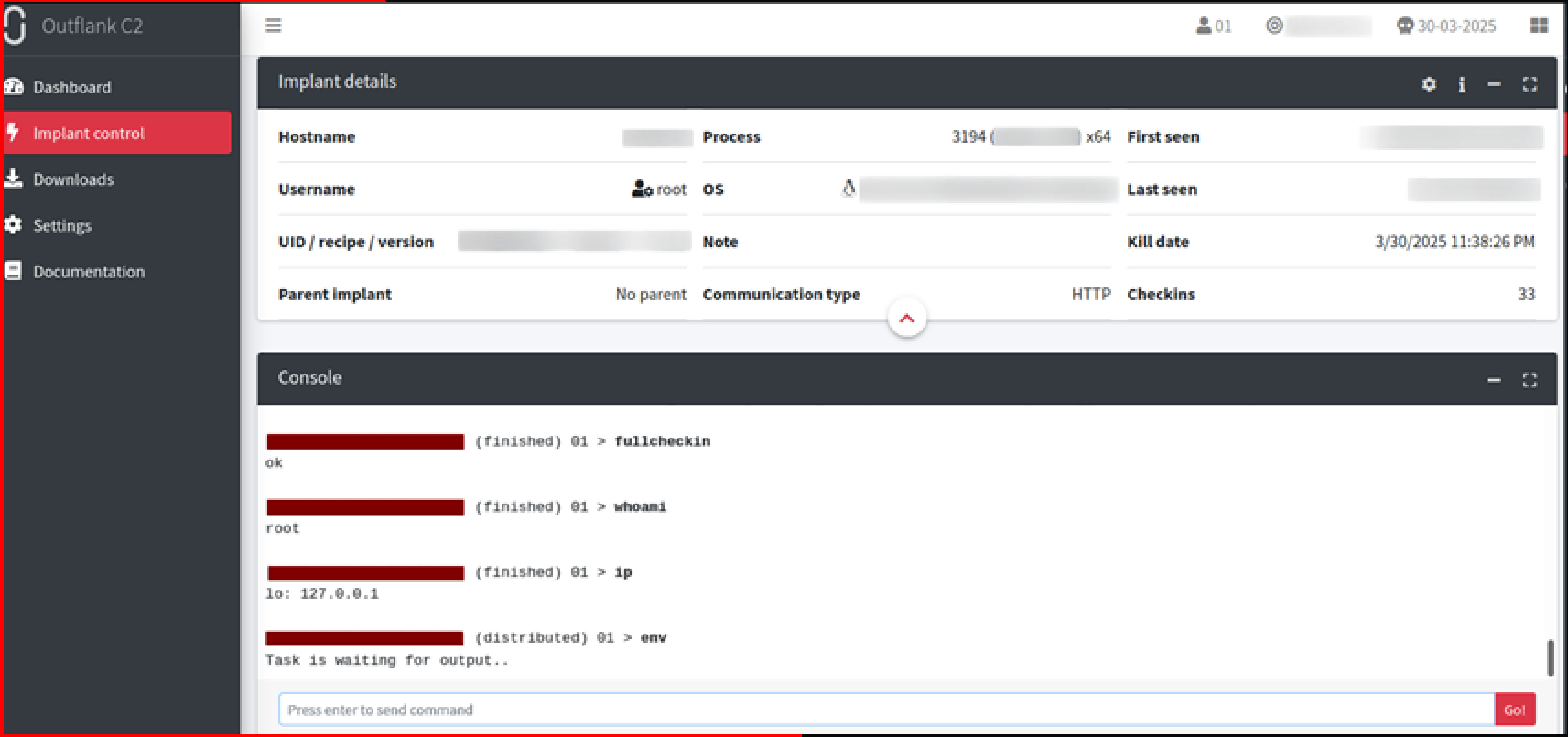


# Exfiltration via HIDX

## PowerShell PoC



HIDX and its power



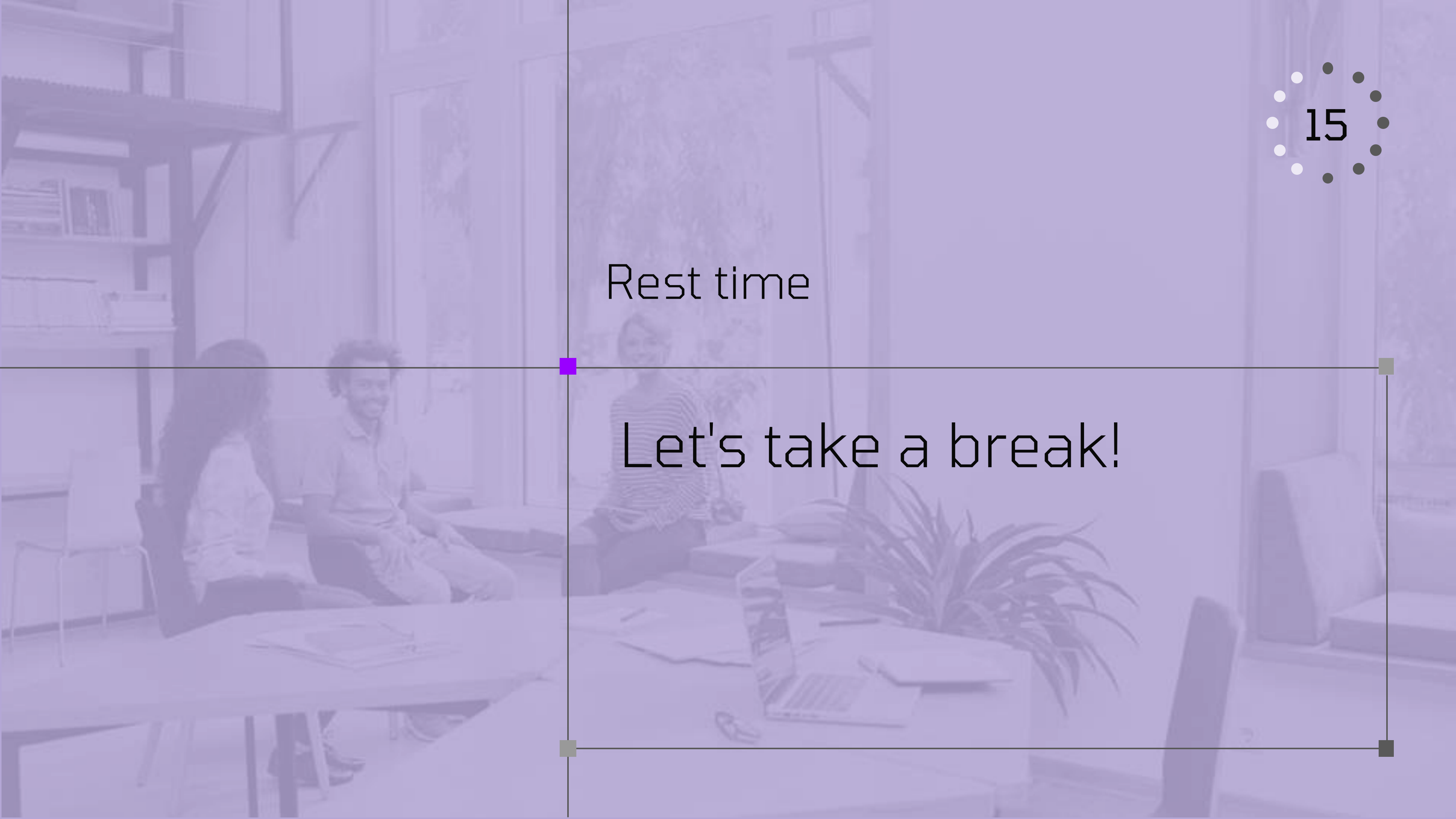
| | | | | | |
|---|---|---|---|---|---|
| **Hostname** | [redacted] | **Process** | 3194 ([redacted]) x64 | **First seen** | [redacted] |
| **Username** | root | **OS** | [redacted] | **Last seen** | [redacted] |
| **UID / recipe / version** | [redacted] | **Note** | | **Kill date** | 3/30/2025 11:38:26 PM |
| **Parent implant** | No parent | **Communication type** | HTTP | **Checkins** | 33 |

```
[redacted] (finished) 01 > fullcheckin
ok

[redacted] (finished) 01 > whoami
root

[redacted] (finished) 01 > ip
lo: 127.0.0.1

[redacted] (distributed) 01 > env
Task is waiting for output..
```

Press enter to send command

# O.MG C2

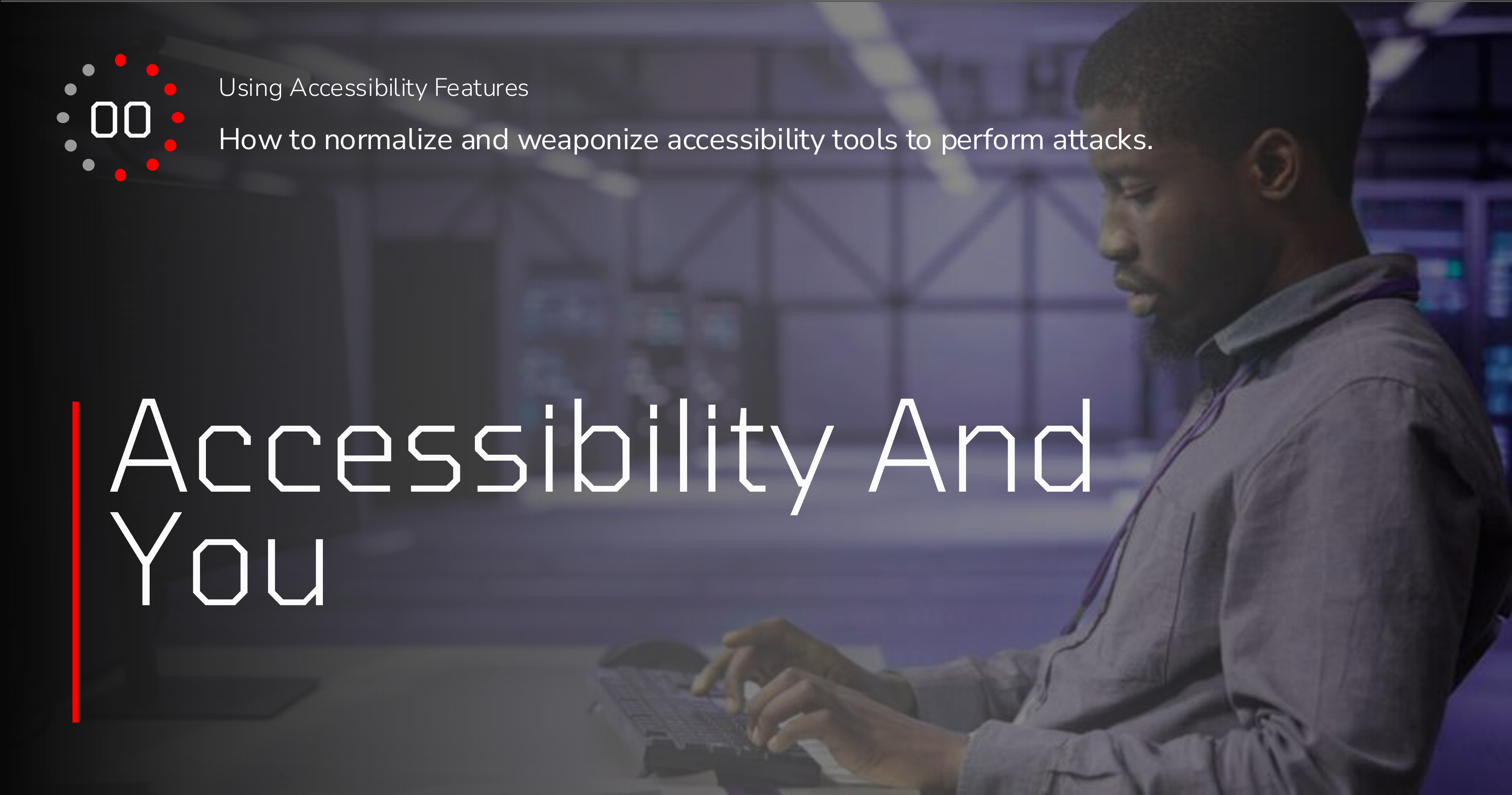
Controlling your fleet:
A very C2 Demo…



15

Rest time

# Let's take a break!



00

Using Accessibility Features

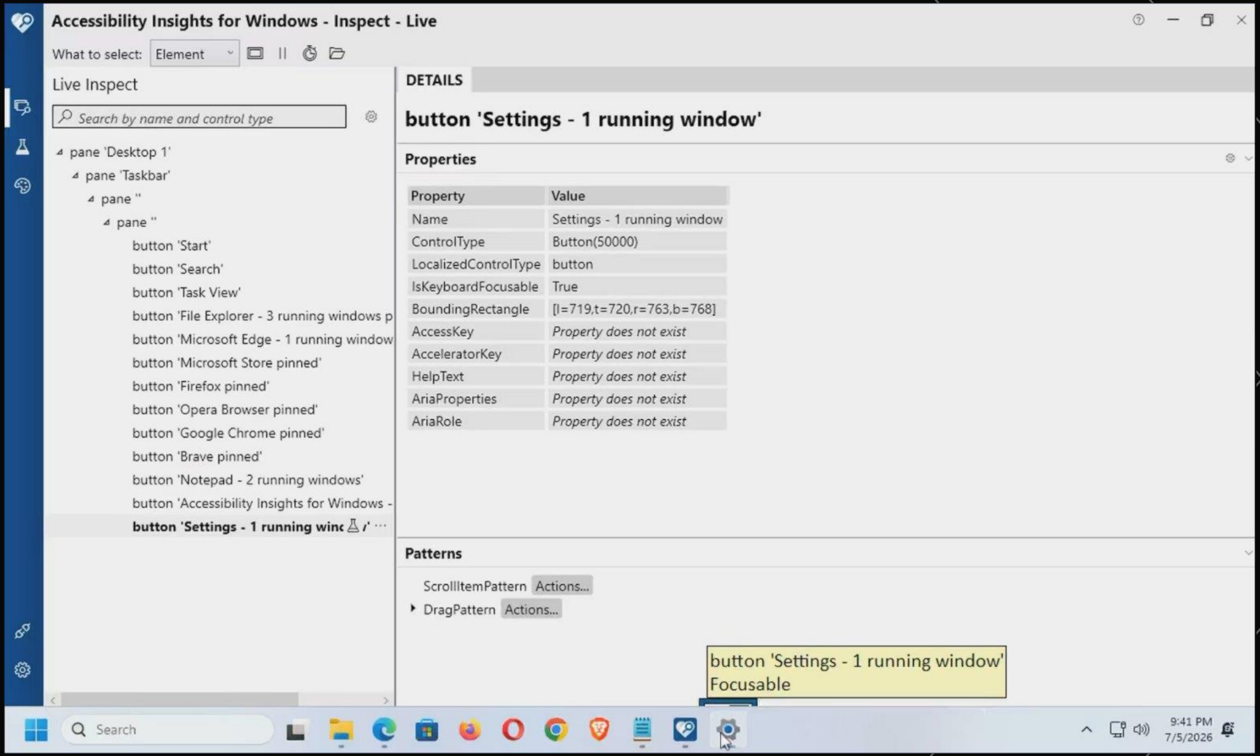
How to normalize and weaponize accessibility tools to perform attacks.

# Accessibility And You

# Accessibility And You

- Understanding how thinking Accessibility First provides better payload design (targeting mouse, locations, positions on screen etC)

- Designing payloads to use keyboard or mouse reliably (screen size independent and device independent



Accessibility Insights for Windows - Inspect - Live

What to select: Element

Live Inspect

Search by name and control type

pane 'Desktop 1'
  pane 'Taskbar'
    pane ''
      pane ''
        button 'Start'
        button 'Search'
        button 'Task View'
        button 'File Explorer - 3 running windows p
        button 'Microsoft Edge - 1 running window
        button 'Microsoft Store pinned'
        button 'Firefox pinned'
        button 'Opera Browser pinned'
        button 'Google Chrome pinned'
        button 'Brave pinned'
        button 'Notepad - 2 running windows'
        button 'Accessibility Insights for Windows -
        **button 'Settings - 1 running winc**

DETAILS

**button 'Settings - 1 running window'**

**Properties**

| Property | Value |
|---|---|
| Name | Settings - 1 running window |
| ControlType | Button(50000) |
| LocalizedControlType | button |
| IsKeyboardFocusable | True |
| BoundingRectangle | [l=719,t=720,r=763,b=768] |
| AccessKey | *Property does not exist* |
| AcceleratorKey | *Property does not exist* |
| HelpText | *Property does not exist* |
| AriaProperties | *Property does not exist* |
| AriaRole | *Property does not exist* |

**Patterns**

ScrollItemPattern Actions...
DragPattern Actions...

button 'Settings - 1 running window'
Focusable

# Any questions?

Ask & Follow us on social media:
@spiceywasabi on Twitter
@01p8or13 on Twitter

# THANK YOU!